

Understanding Context Compression in Agentic Systems

Anonymous ACL submission

Abstract

Agentic systems rely on context compression to execute long-horizon tasks within finite context windows. Existing systems typically adopt a single compression strategy, such as truncation, summarization, or selective eviction, with limited understanding of how these choices affect task success, token usage, and latency across deployment settings. To better understand the impact of these choices, in this paper, we present the first systematic characterization of context compression policies for agentic systems. We organize the compression design space along three design axes: how context is compressed, when compression is triggered, and how aggressively context is compressed. Using this framework, we evaluate 33 compression policies on SWE-bench Verified across multiple context budgets and LLMs. We show that compression policies induce substantially different tradeoffs between task success, token usage, and latency, varying by up to 18 percentage points in task success, $1.5\times$ in token usage, and $1.6\times$ in latency under the same context budget. We further show that different compression policies solve distinct subsets of tasks and that the best-performing policy varies across models and context budgets. Together, our results show that no single compression policy consistently dominates across deployment settings, motivating adaptive compression systems instead of fixed global strategies.

1 Introduction

Agentic systems increasingly execute long-horizon workflows that interleave LLM reasoning, tool calls, code execution, and environment interaction. As trajectories grow over hundreds of steps (Figure 1), accumulated context eventually exceeds finite context windows (Packer et al., 2023), forcing agents to compress prior interactions to continue execution (Xiao et al., 2025; Kang et al., 2025). Long contexts also degrade model effectiveness (Liu et al., 2024; Levy et al., 2024), while

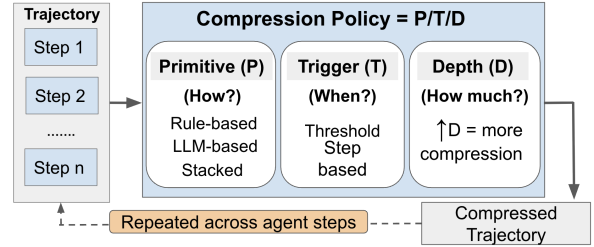


Figure 1: Agent trajectories grow across steps and are repeatedly compressed using policies defined by three axes: compression primitives (P), triggers (T), and depth (D). See § 1.

repeatedly reprocessing accumulated trajectories increases token usage and latency. So, modern agentic systems rely on context compression policies that selectively remove, rewrite, or summarize portions of the trajectory during execution.

Existing systems adopt a wide range of context compression policies. Some agents truncate older interactions entirely (OpenAI, 2025; LangChain, 2025b), while others summarize prior context using auxiliary LLM calls (OpenAI, 2026; Anthropic, 2026). More recent systems combine multiple compression mechanisms, triggering compression at different points during execution and compressing different amounts of context (Packer et al., 2023; Wang et al., 2025). Given this growing design space of context compression policies, it has become difficult to disentangle the impact of individual design decisions, including how agents compress context, when compression is triggered, and how aggressively to compress context, on task success, token consumption, and execution latency.

To this end, we study the compression design space around three design axes: how agents compress context (e.g., truncation versus summarization), when compression is triggered (e.g., compressing only after the context window fills versus compressing after every step), and how aggressively context is compressed (Figure 1). Using this

framework, we evaluate 33 compression policies on SWE-bench Verified using open-source coding agents operating under constrained context budgets. We also study the impact of multiple context budgets and LLMs on the task performance, token usage, and execution latency of tasks.

Our characterization reveals four key findings (§3). First, compression policies induce substantially different tradeoffs between task success, token usage, and execution latency, and no single policy consistently dominates across all three metrics. Second, different compression policies solve overlapping but distinct subsets of tasks. An oracle that selects the best policy per task improves task success by 20 percentage points over the best individual policy, indicating that the choice of compression policy is task-specific. Third, compression policy behavior varies significantly across LLMs, with different models preferring different compression strategies. Fourth, compression policy choice becomes increasingly important as context budgets shrink, since policies differ substantially in how effectively they preserve useful context under aggressive compression.

Together, our results show that context compression is not a single optimization problem with a globally optimal solution. Our findings motivate a shift from evaluating isolated compression strategies toward developing adaptive compression systems that dynamically select policies based on the task, model, and deployment constraints.

We summarize our contributions as follows:

- We present the first systematic characterization of context compression policies for agentic systems, organizing the compression design space along three design axes (§2): how context is compressed, when compression is triggered, and how aggressively context is compressed.
- We evaluate 33 compression policies across SWE-bench Verified under varying compression primitives, context budgets, compression configurations, and models, jointly measuring task success, token usage, and end-to-end latency.
- We show that no single compression policy consistently dominates across tasks, models, and context budgets. Different policies optimize different tradeoffs and solve different subsets of tasks, motivating adaptive compression systems instead of fixed global compression strategies.

2 The Choices in Context Compression

Existing context compression methods (Kang et al., 2025; Xiao et al., 2025; Gu et al., 2026; Anthropic, 2025; OpenAI, 2025; LangChain, 2025a) combine multiple design decisions within a single policy, making it difficult to isolate the impact of individual choices. We represent a context compression policy using three explicit design axes:

1. how the agent compresses context (primitive compression mechanism axis),
2. when the agent triggers compression (trigger axis), and
3. how aggressively the agent compresses context (depth axis).

A compression policy is fully specified by one choice along each axis. This formulation provides a common representation for systematically characterizing context compression behavior across policies. The remainder of this section defines the design space, and Table 1 summarizes all the compression policies evaluated in Section 3.

2.1 Which compression primitive to use?

As agents execute multi-step trajectories, they accumulate tokens across model invocations and tool interactions (Xiao et al., 2025; Kang et al., 2025). Once the trajectory exceeds the available context budget, the agent must compress part of the accumulated context. The compression primitive axis specifies how compression is performed. Different compression primitives preserve different information. For example, truncation removes portions of the trajectory entirely, while summarization replaces them with a compact generated representation. Thus, the choice of primitive determines which information remains available for subsequent reasoning steps.

We categorize compression primitives into three families: rule-based, LLM-based, and stacked. Across these families, we study eight primitives used in existing agentic systems (Table 1). All primitives operate only on the accumulated trajectory context; the system prompt and current task instruction are always preserved verbatim. We describe each family next.

Rule-based. Rule-based primitives compress trajectories using fixed deterministic procedures. We study two rule-based primitives used in production agentic systems:

- *TR* (truncation) removes the oldest messages from the compressible window until the trajec-

tory fits within the token budget (OpenAI, 2025; LangChain, 2025b).

- *TRC* (tool-result clearing) replaces older tool outputs with a short placeholder string (e.g., [output cleared]), while preserving recent tool interactions, assistant reasoning, and user messages (Anthropic, 2025).
- **LLM-based.** LLM-based primitives compress trajectories by generating shorter representations of existing context. We study four summarization primitives that vary along two dimensions: whether summarization is applied to the entire trajectory or only older context, and whether the generated summary is free-form or schema-constrained.
- *SU* (summarization) summarizes the compressible window with a free-form prompt (OpenAI, 2026).
- *SU-p* (partial summarization) applies *SU* only to older portions of the trajectory while preserving recent interactions verbatim (Anthropic, 2026).
- *SS* (structured summarization) summarizes the trajectory using a fixed schema (e.g., files modified, decisions made, or unresolved issues) (Shinn et al., 2023).
- *SS-p* applies *SS* only to older portions of the trajectory while preserving recent interactions verbatim (Packer et al., 2023).

Stacked. Rule-based primitives are computationally inexpensive but can only compress specific portions of the trajectory. LLM-based primitives can compress arbitrary context, but require an additional model invocation at each compression event.

Some production agents combine both approaches to balance these tradeoffs (Packer et al., 2023; Wang et al., 2025). We therefore also study stacked primitives, which first apply a rule-based primitive and invoke an LLM-based primitive only if the trajectory still exceeds the token budget. We denote stacked primitives using an addition operator (e.g., TRC+SU).

2.2 When to trigger context compression?

Compression primitives specify how context is compressed, but not when compression occurs. We study two trigger mechanisms. Under threshold triggers (*Th*), the agent invokes compression only when the accumulated trajectory exceeds a fixed token budget. For example, compression may be triggered once the trajectory grows beyond 15K tokens. Under step triggers (*St*), the agent compresses context after every agent step, such as a tool

invocation or model call, regardless of the current token budget.

2.3 How aggressively should we compress?

This final question determines how aggressively an agent should compress the context window. Formally, to answer this question, we define the *compression depth ratio* as the fraction of the current context removed during compression: $D = 1 - \frac{\text{target tokens}}{\text{current tokens}}$. All prior agents fix this ratio. We explore various settings of $D \in \{0.3, 0.5, 0.7\}$, where larger values denote more compression.

2.4 Formal notation

A compression policy is fully specified by three choices: the compression primitive P , trigger mechanism T , and compression depth D . We denote a policy as $P/T/D$ (e.g., *TRC/St/0.5*). Table 1 enumerates the resulting compression policy design space across all three axes. In total, we study 33 compression policies.

3 Characterization

We now characterize context compression policies across the design space defined in Section 2. We study three questions: how compression policies impact task performance, latency, and token usage; how different compression primitives affect agent behavior across tasks; and whether these trends remain consistent across models.

Benchmark and tasks. We evaluate on SWE-bench Verified (Jimenez et al., 2024), where each task is a real GitHub issue. The resolution is binary, scored by whether the agent’s final patch passes the held-out test patch. Following prior works (Wang et al., 2025; Hou et al., 2024), we sample 100 tasks across the three repositories in SWE-bench Verified (Django 34, SymPy 34, scikit-learn 32).

Agent harness and model. We evaluate compression policies in the mini-swe-agent harness (Yang et al., 2024), a minimal tool-loop agent that exposes a single bash tool with auto-execution and a one-shot tool-call protocol, producing clean turn boundaries for compression. Each task is allocated a maximum of 125 agent steps. We use Qwen3-Coder-30B-A3B (Team, 2025) with temperature 0.2 and served through vLLM on four A100 GPUs. **Experimental setup.** Table 1 summarizes the compression policies evaluated in our study, spanning 8 compression primitives, 2 trigger mechanisms, and 3 compression depths (0.3, 0.5, 0.7). We evaluate

Compress Primitive (P)	Primitive family	Trigger (T)	Depth (D)
TR (OpenAI, 2025; LangChain, 2025b)	Rule-based	threshold	0.3 / 0.5 / 0.7
TRC (Anthropic, 2025)	Rule-based	threshold / step	0.3 / 0.5 / 0.7
SU-full (Anthropic, 2026; OpenAI, 2026; LangChain, 2025a)	LLM-based	threshold	0.3 / 0.5 / 0.7
SU-partial	LLM-based	threshold	0.3 / 0.5 / 0.7
SS (Shinn et al., 2023)	LLM-based	threshold	0.3 / 0.5 / 0.7
SS-partial (Packer et al., 2023)	LLM-based	threshold	0.3 / 0.5 / 0.7
TRC+SU (Anthropic, 2025)	Stacked	threshold / step	0.3 / 0.5 / 0.7
TRC+SS (Anthropic, 2025)	Stacked	threshold / step	0.3 / 0.5 / 0.7

Table 1: Compression policies mapped into the three-axis design space that we study in our characterization. Citations for each primitive link to production runtime strategies. See § 2.

policies under three context budgets (10K, 15K, and 20K tokens) with two runs per task. Our primary setting uses a 15K-token budget, where all primitives are evaluated on the full 100-task benchmark. For primitives with tunable compression depth, we additionally evaluate all three depth settings on the same task set. We use the 10K and 20K settings as budget ablations. Under these settings, depth-invariant primitives are evaluated at depth 0.5 on the full benchmark, while depth-tunable primitives are evaluated across all depths on a 30-task stratified subset. As a baseline, we compare all compression policies against FC (full context), which disables context compression entirely. For reproducibility, we release the task split, model checkpoint, and evaluation harness at TBD¹.

Metrics. We report three metrics: (1) *Resolve rate*, the fraction of tasks successfully solved by the agent (Jimenez et al., 2024); (2) *Token cost*, the total input and output tokens consumed across all model invocations, including compression calls (Kang et al., 2025); and (3) *Latency*, the duration to complete a task from initialization to final solution submission (Ni et al., 2026).

Q1. How do compression policies impact resolve rate, token cost, and latency?

We first analyze the impact of different compression policies on resolve rate, token cost, and latency. Figure 2 shows the tradeoff between token usage and resolve rate (left), and latency and resolve rate (right), under a 15K-token budget. Across the 33 policies, resolve rate varies by 18 percentage points (pp), token usage by $1.5\times$ (462K–716K tokens), and end-to-end latency by $1.6\times$ (563–920 seconds), indicating substantial variation across the compression design space. We next analyze how each design axis—compression primitive, trigger

mechanism, and compression depth—shapes these tradeoffs.

Impact of compression primitives. Figure 2 shows that the three primitive families—rule-based, LLM-based, and stacked—occupy different regions of the tradeoff space. Stacked primitives consistently achieve the highest resolve rates. TRC+SU achieves the best overall performance at 47.0%, followed by TRC+SS at 45.0% and TRC+TR at 44.5%. Rule-based primitives span a wider range: TRC achieves 45.5% resolve rate, while TR drops to 38.5%. LLM-based primitives show the largest variation, ranging from 29.0% for SU to 43.5% for SU-p.

These differences arise from how each primitive family manages context. Rule-based primitives compress context through deletion alone, making them the fastest policies (563–641s) but also the most token-intensive (601K–716K tokens), since they preserve larger portions of the original trajectory. LLM-based primitives instead rewrite context through additional model calls, reducing token usage (487K–583K tokens) at the cost of substantially higher latency (734–920s). Their resolve rates also vary widely, indicating that summary quality depends heavily on which information is preserved during rewriting. Stacked primitives combine both approaches by first pruning context using rule-based compression before invoking summarization only when necessary. This reduces the amount of context processed by the summarization model, resulting in intermediate token usage (462K–644K tokens) and latency (625–774s) while consistently achieving the highest resolve rates.

Impact of compression triggers. To isolate the impact of compression triggers, we compare TRC under threshold versus per-step triggers (§ 2). We only analyze TRC here; compression per step with LLM-based methods is prohibitively expensive, while TR at per-step granularity is overly aggressive and sig-

¹Link redacted to ensure anonymity during review process

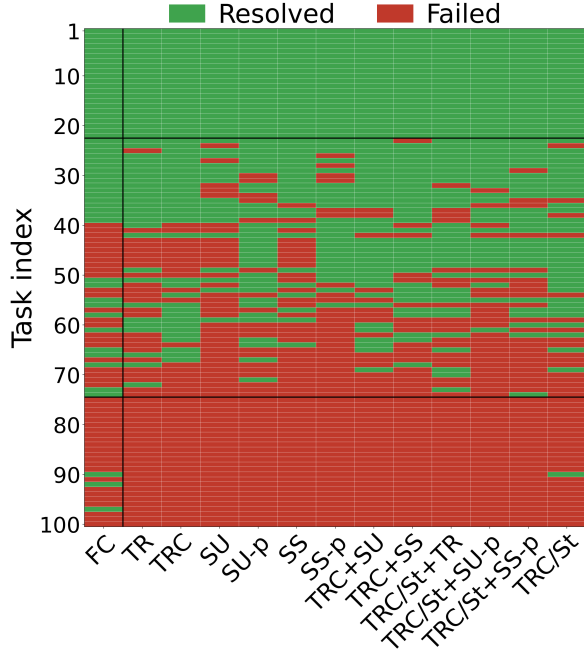


Figure 4: Per-task outcomes (rows) across 12 compression policies (columns) at a 15K-token budget and depth $D = 0.5$. Green indicates resolved tasks and red unresolved tasks. The mixed middle region shows outcomes depend strongly on the compression policy. See § 3 Q2.

and compression depth must be tuned jointly with the primitive. Thus, *compression policies require axis-specific tuning rather than a single globally optimal configuration*.

Q2. Does the choice of compression policy affect which tasks an agent solves?

So far, we compared compression policies using aggregate resolve rates across all tasks. This comparison implicitly assumes that stronger policies simply solve a superset of the tasks solved by weaker ones. We find that this assumption does not hold.

Figure 4 shows the per-task outcomes for all compression policies. Across the benchmark, 22% of tasks are solved by every policy, while 25% are solved by none. For the remaining 53% of tasks, however, task resolution depends strongly on the chosen compression policy. An oracle that selects the best policy per task achieves a 67% resolve rate, improving by 20 percentage points over the best individual policy (TRC+SU at 47%).

Different tasks require preserving different forms of information across long trajectories. Compression policies vary substantially in which information they retain. For example, TR preserves recent context verbatim while discarding older interactions entirely. SU rewrites the trajectory into a compact free-form summary, compressing ag-

gressively but potentially removing structural details. SS instead preserves information within a fixed schema, while step-triggered TRC policies aggressively remove prior tool outputs throughout execution. As a result, different policies produce different trajectory representations, causing different tasks to favor different compression behaviors. **Takeaway #2.** Compression policies solve overlapping but distinct subsets of tasks. So, *the best compression policy is task-dependent, since policies preserve different information from the trajectory*.

Q3. How does the efficacy of compression policies vary across models and token budgets?

In Q1 and Q2, we fixed the model, context budget, and model precision following prior work (Kang et al., 2025; Xiao et al., 2025). We next study how compression policy behavior changes as these deployment settings vary. We next evaluate three open-source coding models—Qwen3.5-35B-A3B-Instruct, Devstral-Small-24B, and Qwen2.5-Coder-32B-Instruct—across three context budgets (10K, 15K, and 20K tokens) and four model precisions (fp16, fp8, GPTQ-int4, and AWQ-int4). Each configuration is evaluated on a 30-task stratified subset. We fix the compression depth to $D=0.5$, which is the default setting used throughout Q1 and Q2. Since Section 3 already studies depth as an independent design axis, fixing D here isolates the effects of model choice and context budget from depth-dependent variation. We analyze the effects of model and context budget in this section and defer precision results to Appendix 4.

Impact of models. Across models, the best compression policy changes. While models exhibit the same tradeoff between resolve rate, token usage, and latency (Q1), the policies that optimize these tradeoffs differ across models. For Qwen3.5-35B-A3B-Instruct, $TRC + SU/Th$ achieves the highest resolve rate (47%), while TRC/Th performs best on Devstral-Small-2-24B (68.3%) and $TRC + TR/St$ performs best on Qwen2.5-Coder-32B-Instruct (11.7%). These differences are also large in magnitude: on Devstral-Small-2-24B, TRC/Th outperforms $TRC + SU/Th$, the best policy for Qwen3.5-35B-A3B-Instruct, by roughly 15pp, while $TRC + SU/Th$ outperforms TRC/Th on Qwen3.5-35B-A3B-Instruct by 2.5pp. More broadly, policy rankings shift across models. The ranking correlation between Qwen3.5-35B-A3B-Instruct and Devstral-Small-

Table 2: Resolve rate, token cost, and latency across token budgets for 30 tasks on SWE-bench Verified. The LLM is fixed to Qwen3.5-35B-A3B, and compression depth is fixed at $D = 0.5$. See § 3 Q3.

Primitive	10K			15K			20K		
	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)
TR	48.3	525	398	33.3	748	601	60.0	764	474
TRC	46.7	461	368	51.7	545	417	71.7	662	434
SU-full	31.7	424	915	23.3	587	801	56.7	685	556
SU-partial	40.0	370	949	56.7	566	785	61.7	628	737
SS	35.0	274	984	30.0	497	707	61.7	657	711
SS-partial	36.7	273	978	53.3	524	794	65.0	587	750
TRC+SU	55.0	475	637	65.0	580	638	66.7	681	732
TRC+SS	60.0	438	669	58.3	540	632	76.7	669	653
TRC/St+TR	46.7	437	511	55.0	501	572	66.7	517	555
TRC/St+SU-partial	48.3	321	750	65.0	460	617	61.7	495	606
TRC/St+SS-partial	38.3	372	890	51.7	393	656	65.0	474	596

2-24B is moderate (Spearman 0.61), and drops to 0.14 between Qwen3.5-35B-A3B-Instruct and Qwen2.5-Coder-32B-Instruct.

We also find that the benefits of compression relative to full context (FC) depend strongly on the model. For Qwen3.5-35B-A3B-Instruct, six compression policies outperform FC in resolve rate, token usage, and latency (see Table 3). In contrast, for Devstral-Small-24B, the best compression policy (TRC/Th) reduces token usage by $1.7\times$ but resolves 3.3pp fewer tasks while increasing latency by $1.4\times$. For Qwen2.5-Coder-32B-Instruct, even the tradeoff between token usage and resolve rate largely disappears: the best compressed policy ($TRC + TR/St$) improves resolve rate only marginally over FC (11.7% vs. 10.0%) while increasing both token usage and latency. These results show that compression policy behavior is strongly model-dependent. For some models, compression improves both efficiency and task performance. For others, compression introduces tradeoffs between resolve rate, token usage, and latency.

Takeaway #3. *The efficacy of compression policies shifts across models:* the optimal policy and benefit of compression relative to no compression vary across models and capabilities.

Impact of token budgets. Unlike the impact of models, we observe less fluctuation in relative rankings across token budgets. Generally, the rankings of compression primitive families remain the same: stacked, rule-based, and finally LLM-based. Moreover, the best and worst primitive is the same across token budgets (a threshold-stacked primitive and SU, respectively). However, within families, there are slight fluctuations. Hence, the Spearman rank

correlation between 10K and 15K token budget is 0.7, while between 20K and 15K it is 0.8.

The benefits of compression relative to full context (FC) also vary with token budget. At larger budgets (20K), several compression policies outperform FC in resolve rate, token usage, and latency. At smaller budgets (10K), however, FC achieves higher resolve rates. Larger budgets allow compression policies to remove redundant context while preserving information needed for downstream reasoning. Meanwhile, smaller budgets force compression policies to discard increasingly useful context from shorter trajectories. As a result, compression consistently reduces token usage, but increasingly trades off resolve rate at tighter budgets.

Takeaway #4. Context compression policy rankings remain relatively stable across context budgets. However, *compression policy choice matters most at smaller budgets, where policies increasingly differ in how effectively they preserve useful context under aggressive compression.*

4 Implications for Future Agentic Systems

Our findings show that context compression behavior is far more structured and task-specific than existing systems assume. We next discuss the broader implications of these results for future agentic systems and context compression research.

Adaptive compression systems. Existing agentic systems (OpenAI, 2026; Anthropic, 2026; LangChain, 2025b; Wang et al., 2025) typically deploy a single fixed compression policy across all tasks and deployment settings. Our characterization shows that no single policy consistently

Table 3: Resolve rate, token cost, and latency across models for 30 tasks on SWE-bench Verified. The token budget is fixed at 15K tokens and compression depth at $D = 0.5$. See § 3 Q3.

Primitive	Qwen3.5-35B-A3B			Devstral-Small-2-24B			Qwen2.5-Coder-32B		
	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)
FC	43.0	1020	737	71.7	1232	290	10.0	437	566
TR	38.5	716	641	63.3	814	451	10.0	512	672
TRC	45.5	601	563	68.3	738	403	8.3	618	856
SU-full	29.0	583	812	43.3	968	727	8.3	348	846
SU-partial	43.5	572	821	43.3	646	1192	8.3	358	871
SS	36.0	487	734	30.0	911	714	6.7	351	930
SS-partial	38.0	558	920	38.3	743	938	8.3	290	869
TRC+SU	47.0	644	768	50.0	833	427	6.7	558	777
TRC+SS	45.0	607	727	53.3	892	486	11.7	508	742
TRC/St+TR	44.5	555	625	48.3	595	537	11.7	554	556
TRC/St+SU-partial	39.5	500	774	36.7	610	737	6.7	485	746
TRC/St+SS-partial	42.0	462	733	38.3	598	642	6.7	433	693

dominates across tasks, models, and context budgets. Different policies preserve different information and succeed on different subsets of tasks. This motivates adaptive compression systems that dynamically select compression policies based on the task, model behavior, available context budget, and execution state. A natural next step is to learn compression controllers from workload features, task planning, intermediate trajectory state, thus turning compression policy selection into an online systems optimization problem for agentic systems.

Context compression as a systems resource-management problem. All prior work (Kang et al., 2025; Xiao et al., 2025; Gu et al., 2026) studies context compression primarily as a prompting or representation problem. Our findings show that compression policies also induce large systems-level tradeoffs across token usage, latency, and task success. So, compression decisions directly shape the operational behavior of agentic systems under constrained context budgets. This suggests that future work should treat context compression as a systems resource-management problem, jointly optimizing compression behavior alongside inference cost, scheduling, and deployment constraints.

Beyond aggregate evaluation. All existing work (Kang et al., 2025; Xiao et al., 2025; Gu et al., 2026) evaluates context compression using aggregate task success under a fixed model and deployment setting. Our characterization shows that aggregate metrics obscure substantial structure in compression behavior. Policies with similar aggregate performance often solve different subsets of tasks, and policy rankings shift across models and

context budgets. Hence, future evaluations should move beyond single aggregate metrics and instead characterize compression behavior across tasks, deployment settings, and operating constraints.

Compression-aware agent design. Existing agents (Yang et al., 2024; Wang et al., 2025) generate trajectories without considering how those trajectories will later be compressed. Our characterization shows that different compression policies preserve different forms of information and alter which tasks agents successfully complete. Future agents may therefore need to explicitly structure trajectories for downstream context compression. For example, agents may separate persistent state from ephemeral reasoning, expose semantic checkpoints for summarization, or generate compression-friendly execution traces. These findings motivate jointly designing agents and context compression systems rather than treating compression as an external add-on applied after trajectory generation.

5 Conclusion

We presented the first systematic characterization of context compression policies for agentic systems. Our study shows that compression policies induce substantially different tradeoffs across task success, token usage, and latency, solve different subsets of tasks, and vary significantly across models and context budgets. These findings demonstrate that no single compression policy consistently dominates across deployment settings. More broadly, our results motivate adaptive compression systems that dynamically select compression policies based on the task, model, and context budget.

6 Limitations

Our study characterizes the cost-resolve trade-off of context-compression primitives across three open-weight models, four token budgets, and three compression depths. While our results give a consistent picture of how primitives behave inside this regime, the current study has a few limitations.

First, we run all primitives inside a single agent harness, mini-swe-agent. The harness uses a trajectory-style tool-use loop with a fixed tool set, which matches the most common deployment-style agent design (Jimenez et al., 2024; Wang et al., 2025). Holding the harness fixed exercises every primitive under identical conditions and isolates compression-strategy effects from harness-design effects. Harnesses that use richer memory architectures, planner-executor splits, or substantially different tool sets may exhibit different compression behavior, and we have not measured this transfer directly.

Second, we evaluate on three open-weight code-capable models in the 24–35B range, drawn from three different lineages (Qwen3, Devstral, Qwen2.5-Coder). This range balances coverage across distinct model families with the compute needed to run every primitive at every budget. The three lineages let us probe transfer of the trade-off across qualitatively different agents while keeping the full primitive sweep tractable. We have not measured frontier closed-source models, much smaller models, or models trained with explicit long-context curricula, and the operating points reported here may not extrapolate to those regimes.

Third, our summarization-based primitives use the same model as the agent itself to compress the trajectory. While this keeps the deployment footprint small and isolates the effect of compression strategy from summarizer-quality differences, a separately trained or stronger summarizer could change the ranking of summarization-based primitives, and we have not measured this dimension.

Fourth, several internal defaults of our compression primitives are fixed at single values rather than swept along their own axes. Our TRC variants preserve the three most recent tool-result turns, our summarization-based primitives use a single hand-written instruction prompt for both SU and SS, and the protected prefix of every trajectory is fixed at the system message and the first user turn. While these defaults are consistent across all of our runs and isolate the effect of the primitive choice itself,

each of them can shift resolve and cost within a primitive family, and we have not measured the sensitivity of our results to these knobs.

Addressing these dimensions; the agent harness, the summarizer-model coupling, and the internal defaults, would strengthen the generality of the cost-resolve trade-off characterized here, and we leave this to future work.

References

- Anthropic. 2025. Context editing. <https://platform.claude.com/docs/en/build-with-claude/context-editing>. Accessed 2026-05-11.
- Anthropic. 2026. Compaction. <https://platform.claude.com/docs/en/build-with-claude/compaction>. Accessed 2026-05-11.
- Zhuohan Gu, Qizheng Zhang, Omar Khattab, and Samuel Madden. 2026. Peek: Context map as an orientation cache for long-context llm agents. *arXiv preprint arXiv:2605.19932*.
- Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.
- Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2025. Acon: Optimizing context compression for long-horizon llm agents. *arXiv preprint arXiv:2510.00615*.
- LangChain. 2025a. Short-term memory. <https://docs.langchain.com/oss/python/langchain/short-term-memory>. Accessed: 2026-05-11.
- LangChain. 2025b. Short-term memory — trim messages. <https://docs.langchain.com/oss/python/langchain/short-term-memory>. Accessed 2026-05-11.
- Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15339–15353.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paran-jape, Michele Bevilacqua, Fabio Petroni, and Percy

Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173.

Kangqi Ni, Wenyue Hua, Xiaoxiang Shi, Jiang Guo, Shiyu Chang, and Tianlong Chen. 2026. Chimera: Latency-and performance-aware multi-agent serving for heterogeneous llms. *arXiv preprint arXiv:2603.22206*.

OpenAI. 2025. Responses api — create response. <https://platform.openai.com/docs/api-reference/responses/create>. Accessed 2026-05-11.

OpenAI. 2026. Compaction. <https://developers.openai.com/api/docs/guides/compaction>. Accessed 2026-05-11.

Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.

Qwen Team. 2025. *Qwen3 technical report*. Preprint, arXiv:2505.09388.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, and 1 others. 2025. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919.

Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2025. Improving the efficiency of llm agent systems through trajectory reduction. *arXiv preprint arXiv:2509.23586*.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.

A Example Appendix

Across Quantizations. We next test whether the compression conclusion survive lower precision serving. We run a subset of the primitives, namely FC, TR, SU, and TRC+SS, each representing one of the primitive families. Together, these policies span the main families in our taxonomy (§2). We evaluate Qwen3-30B-A3B-Instruct at four precisions: fp16, fp8, gptq-int4 (), and awq-int4 (). Compressed policies use a 20k budget and depth 0.5.

Each condition uses two runs per task, yielding 240 trajectories per precision.

The results split cleanly into two regimes. fp16 and fp8 behave similarly, while both 4-bit quantizations degrade sharply. Resolve rate is 9.2% at fp16 and 7.1% at fp8, but drops to 3.3% with GPTQ-int4 and 2.9% with AWQ-int4. Because every task and policy is run under each precision setting, the comparison controls for task difficulty and policy choice. Thus, differences in resolve rate can be attributed more directly to precision rather than to different task samples. Token use is broadly comparable across precision, with median tokens per run between 32k and 40k.

The Q2 heterogeneity check is not informative on this base model. Even at fp16, the model solves too few tasks for policy differences to be meaningfully analyzed: 80% of tasks are not solved by any primitive, leaving only a small set of tasks where the choice of compression strategy can affect the outcome. Under 4-bit quantization this becomes even more severe, with 90% of tasks unsolved by all primitives.

Takeaway. The Q1 trade-off transfers from fp16 to fp8, but not to 4-bit. At 4-bit, the base model resolves too few tasks for compression policy choice to matter, so the strategy-level contrasts largely disappear.

Table 4: Resolve rate, token cost, and latency for Qwen3-30B-A3B-Instruct-2507 across model precision (quantization) types. Token budget is fixed to 20K and compression depth $D = 0.5$. See Appendix A.

Primitive	FP16			FP8			GPTQ-int4			AWQ-int4		
	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)	Res%	Tok(K)	Lat(s)
FC	11.7	240	157	5.0	59	890	3.3	176	514	3.3	57	859
TR	10.0	404	254	10.0	53	800	0.0	169	580	5.0	91	808
SU-full	8.3	239	224	3.3	56	877	5.0	131	555	3.3	61	868
TRC+SS	6.7	314	325	10.0	70	855	5.0	230	633	0.0	74	861